

Name: M Venkata Sai

Register Number: 192425223

CO3 – AT2

MLA 0201 – Fundamentals of Machine Learning

Annexure A

Case Study

Case Study 1: Customer Segmentation using K-Means Clustering and PCA

A retail store wants to segment its customers based on their Annual Income and Spending Score to design targeted marketing campaigns. Using the dataset below, write a Python program to preprocess the data, apply K-Means clustering, reduce the data to two dimensions using Principal Component Analysis (PCA), visualize the clusters, and identify the optimal number of customer groups.

Dataset (customer_segmentation.csv)

CustomerID Age AnnualIncome (k\$) SpendingScore

1 19 15 39

2 21 15 81

3 20 16 6

4 23 16 77

5 31 17 40

6 22 17 76

7 35 18 6

8 23 18 94

9 64 19 3

10 30 19 72

Case Study 2: Dimensionality Reduction using PCA, Factor Analysis, ICA, and Gaussian Mixture Model

A wine manufacturer has collected chemical measurements of different wine samples and wants to reduce the dataset dimensions before performing clustering. Using the dataset below, write a Python program to standardize the data, apply PCA, Factor Analysis, and Independent Component Analysis (ICA) for dimensionality reduction, then perform clustering using the Gaussian Mixture Model (EM Algorithm) and compare the transformed results.

Dataset (wine_samples.csv)

Sample Alcohol MalicAcid Ash Alcalinity Magnesium Phenols

1	14.23	1.71	2.43	15.6	127	2.80
2	13.20	1.78	2.14	11.2	100	2.65
3	13.16	2.36	2.67	18.6	101	2.80
4	14.37	1.95	2.50	16.8	113	3.85
5	13.24	2.59	2.87	21.0	118	2.80
6	14.20	1.76	2.45	15.2	112	3.27
7	14.39	1.87	2.45	14.6	96	2.50
8	14.06	2.15	2.61	17.6	121	2.60
9	14.83	1.64	2.17	14.0	97	2.80
10	13.86	1.35	2.27	16.0	98	2.98

Case Study 1: Customer Segmentation using K-Means + PCA

Objective

We will:

1. Create/load the customer dataset.
2. Select **Annual Income** and **Spending Score**.
3. Standardize the data.
4. Find the optimal number of clusters using the **Elbow Method**.
5. Apply **K-Means Clustering**.
6. Apply **PCA** to reduce the data to 2 dimensions.

7. Visualize the customer groups.
8. Display the cluster assigned to each customer.

CASE STUDY 1

6. Apply PCA

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
df["PCA1"] = X_pca[:, 0]
```

```
df["PCA2"] = X_pca[:, 1]
```

```
print("\nExplained Variance Ratio:")
```

```
print(pca.explained_variance_ratio_)
```

```
print("\nTotal Explained Variance:",
```

```
      sum(pca.explained_variance_ratio_))
```

7. Visualize clusters after PCA

```
plt.figure(figsize=(8, 6))
```

```
for cluster in range(optimal_k):
```

```
cluster_data = df[df["Cluster"] == cluster]
```

```
plt.scatter(  
    cluster_data["PCA1"],  
    cluster_data["PCA2"],  
    label=f"Cluster {cluster}"  
)
```

```
plt.xlabel("Principal Component 1")  
plt.ylabel("Principal Component 2")  
plt.title("Customer Segmentation using K-Means + PCA")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
# -----  
# 8. Display final results  
# -----
```

```
print("\nFinal Customer Segmentation:")
```

```
print(  
    df[  
        [  
            "CustomerID",  
            "Age",  
            "AnnualIncome",  
            "SpendingScore",  
            "Cluster"  
        ]  
    ]
```

```

    ]
)

# -----
# 9. Display number of customers in each cluster
# -----

print("\nCustomers in Each Cluster:")
print(df["Cluster"].value_counts().sort_index())

```

Explanation

Preprocessing:

Annual Income and Spending Score are selected because they are the features required for customer segmentation. Standardization makes both features comparable.

K-Means:

K-Means divides customers into groups based on similarity. Customers having similar income and spending behavior are placed in the same cluster.

Elbow

Method:

We calculate the inertia for different values of K. The point where the decrease in inertia starts becoming smaller indicates a suitable number of clusters.

PCA:

PCA transforms the original features into principal components. Here, we reduce the standardized data to two dimensions so that the clusters can be visualized.

Expected

interpretation:

The clusters represent different customer types, such as:

- Lower income / low spending
- Lower income / high spending
- Higher spending or relatively different spending behavior

Because this dataset contains only **10 customers**, the exact cluster pattern is less representative than it would be with a larger real-world customer dataset.

Case Study 2: PCA + Factor Analysis + ICA + Gaussian Mixture Model

Objective

We will:

1. Create the wine dataset.
2. Standardize the chemical measurements.
3. Apply **PCA**.
4. Apply **Factor Analysis**.
5. Apply **Independent Component Analysis (ICA)**.
6. Apply **Gaussian Mixture Model (GMM)** clustering.
7. Compare the transformed results.
8. Visualize the three dimensionality-reduction techniques.

CASE STUDY 2

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 9. Visualize PCA
```

```
# -----
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(
```

```
    X_pca[:, 0],
```

```
    X_pca[:, 1]
```

```
)
```

```
plt.xlabel("Principal Component 1")
```

```
plt.ylabel("Principal Component 2")
```

```
plt.title("PCA Dimensionality Reduction")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 10. Visualize Factor Analysis
```

```
# -----
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
    X_fa[:, 0],  
    X_fa[:, 1]  
)
```

```
plt.xlabel("Factor 1")
```

```
plt.ylabel("Factor 2")
```

```
plt.title("Factor Analysis")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# -----
```

```
# 11. Visualize ICA
```

```
# -----
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(  
    X_ica[:, 0],  
    X_ica[:, 1]  
)
```

```
plt.xlabel("Independent Component 1")
plt.ylabel("Independent Component 2")
plt.title("Independent Component Analysis")
plt.grid(True)
plt.show()
```

```
# -----
# 12. Compare transformed results
# -----
```

```
comparison = pd.DataFrame({
    "Sample": df["Sample"],
    "PCA_1": X_pca[:, 0],
    "PCA_2": X_pca[:, 1],
    "FA_1": X_fa[:, 0],
    "FA_2": X_fa[:, 1],
    "ICA_1": X_ica[:, 0],
    "ICA_2": X_ica[:, 1],
    "GMM_Cluster": gmm_labels
})
```

```
print("\nComparison of Dimensionality Reduction Methods:")
print(comparison)
```

Small correction for the PCA + GMM plot

To make the program cleaner, you can directly use `X_pca`:

```
plt.figure(figsize=(8, 6))
```

```
for cluster in range(3):
```



```
indices = gmm_labels == cluster
```

```
plt.scatter(  
    X_pca[indices, 0],  
    X_pca[indices, 1],  
    label=f"Cluster {cluster}"  
)
```

```
plt.xlabel("PCA Component 1")  
plt.ylabel("PCA Component 2")  
plt.title("GMM Clustering on PCA-Reduced Data")  
plt.legend()  
plt.grid(True)  
plt.show()
```

Important theory for your viva

Technique	Main purpose	Output
PCA	Finds directions of maximum variance	Principal Components
Factor Analysis	Finds hidden/latent factors	Factors
ICA	Finds statistically independent components	Independent Components
K-Means	Groups similar data points	Hard cluster labels
GMM / EM	Probabilistic clustering	Cluster probabilities
StandardScaler	Makes features comparable	Standardized features

Case Study 1 flow

Dataset → Preprocessing → StandardScaler → Elbow Method → K-Means → PCA → Visualization → Customer Groups

Case Study 2 flow

Dataset → StandardScaler → PCA / Factor Analysis / ICA → Reduced Features → GMM (EM) → Clusters → Comparison

